

6. Develop a program to sort the contents of a text file and write the sorted contents into a separate text file. [Hint: Use string methods strip(), len(), list methods sort(), append(), and file methods open(), readlines(), and write()].

```
def sort_file_contents(input_file, output_file):
```

```
    # Open the input file in read mode
```

```
    with open(input_file, 'r') as file:
```

```
        # Read the contents of the file
```

```
        contents = file.readlines()
```

```
        # Strip the contents of each line
```

```
        contents = [line.strip() for line in contents]
```

```
        # Sort the contents in ascending order
```

```
        contents.sort(key=len)
```

```
        # Write the sorted contents to the output file
```

```
    with open(output_file, 'w') as file:
```

```
        for line in contents:
```

```
            file.write(line + '\n')
```

```
# call function
```

```
sort_file_contents('input.txt', 'output.txt')
```

In this code, the `sort_file_contents` function takes two parameters: the name of the input file, and the name of the output file. The function uses the `open` method to open the input file for reading, reads its contents into a list of lines using the `readlines` method, sorts the list using the `sort` method, and opens the output file for writing. The sorted lines are then written to the output file using the `write` method, with the `strip` method used to remove any leading or trailing whitespace from each line. The resulting output file will contain the sorted contents of the input file.

Prerequisites: input.txt text file with contents in the same folder where program file is saved.

```
def sort_file_contents(input_file_name, output_file_name):
    # Open the input file for reading
    with open(input_file_name, 'r') as input_file:
        # Read the contents of the file into a list
        lines = input_file.readlines()

    # Sort the list of lines
    lines.sort(key=len)

    # Open the output file for writing
    with open(output_file_name, 'w') as output_file:
        # Write the sorted lines to the output file
        for line in lines:
            output_file.write(line.strip() + '\n')

# Call the sort_file_contents function with the names of the input and output files
sort_file_contents('input.txt', 'output.txt')
```

7. Develop a program to backing Up a given Folder (Folder in a current working directory) into a ZIP File by using relevant modules and suitable methods.

```
import os

import zipfile

def backup_folder(folder_name):

    # get the absolute path of the folder

    folder_path = os.path.abspath(folder_name)

    # create a zip file with the same name as the folder

    backup_file = folder_name + '.zip'

    # open the zip file for writing

    with zipfile.ZipFile(backup_file, 'w') as zf:

        # iterate over all files in the folder

        for root, dirs, files in os.walk(folder_path):

            for file in files:

                # get the file path

                file_path = os.path.join(root, file)

                # add the file to the zip file

                zf.write(file_path)

# call the backup_folder function with the name of the folder you want to backup

backup_folder('example_folder')
```

This program uses the os and zipfile modules to create a ZIP file with the same name as the folder you want to backup. The os.walk method is used to iterate over all files in the folder and its subdirectories, and the zipfile.ZipFile.write method is used to add each file to the ZIP file. The resulting ZIP file will contain the entire folder and its contents, preserving the directory structure.

```
import os
import zipfile

def backup_folder(folder_name):
    # get the absolute path of the folder
    folder_path = os.path.abspath(folder_name)

    # create a zip file with the same name as the folder
    backup_file = folder_name + '.zip'

    # open the zip file for writing
    with zipfile.ZipFile(backup_file, 'w') as zf:
        # iterate over all files in the folder
        for root, dirs, files in os.walk(folder_path):
            for file in files:
                # get the file path
                file_path = os.path.join(root, file)
                # add the file to the zip file
                zf.write(file_path)

# call the backup_folder function with the name of the folder you want to backup
backup_folder('example_folder')
```

8. Write a function named DivExp which takes TWO parameters a, b and returns a value c ($c=a/b$). Write suitable assertion for $a>0$ in function DivExp and raise an exception for when $b=0$. Develop a suitable program which reads two values from the console and calls a function DivExp

```
def DivExp(a, b):  
  
    # Check that the denominator is not zero  
  
    if b == 0:  
  
        raise ValueError("Denominator cannot be zero")  
  
    # Check that the numerator is positive  
  
    assert a > 0, "Numerator must be positive"  
  
    # Perform division and return the result  
  
    return a / b  
  
# Read input values from the console  
  
a = float(input("Enter the numerator: "))  
  
b = float(input("Enter the denominator: "))  
  
# Call the DivExp function and print the result  
  
try:  
  
    result = DivExp(a, b)  
  
    print(f"The result of the division is: {result}")  
  
except ValueError as e:  
  
    print(e)
```

In this code, the DivExp function checks that b is not zero, and raises a ValueError exception if it is. It also checks that a is positive using an assert statement. If the assert statement fails, an AssertionError exception will be raised. The main part of the program reads the values of

a and b from the console, calls the DivExp function, and handles any exceptions that may be raised.

```
def DivExp(a, b):  
    # Check that the denominator is not zero  
    if b == 0:  
        raise ValueError("Denominator cannot be zero")  
  
    # Check that the numerator is positive  
    assert a > 0, "Numerator must be positive"  
  
    # Perform division and return the result  
    return a / b  
  
# Read input values from the console  
a = float(input("Enter the numerator: "))  
b = float(input("Enter the denominator: "))  
  
# Call the DivExp function and print the result  
try:  
    result = DivExp(a, b)  
    print(f"The result of the division is: {result}")  
except ValueError as e:  
    print(e)
```

9. Define a function which takes TWO objects representing complex numbers and returns new complex number with a addition of two complex numbers. Define a suitable class 'Complex' to represent the complex number. Develop a program to read N (N >=2) complex numbers and to compute the addition of N complex numbers.

```
class Complex:
    def __init__(self, real, imag):
        self.real = real
        self.imag = imag

    def __str__(self):
        return f"{self.real} + {self.imag}i"

    def __add__(self, other):
        return Complex(self.real + other.real, self.imag + other.imag)

def add_complex_numbers(complex_numbers):
    result = complex_numbers[0]
    for i in range(1, len(complex_numbers)):
        result = result + complex_numbers[i]
    return result

# Read the number of complex numbers
n = int(input("Enter the number of complex numbers: "))

# Read the complex numbers
complex_numbers = []
for i in range(n):
    real = float(input(f"Enter the real part of complex number {i + 1}: "))
    imag = float(input(f"Enter the imaginary part of complex number {i + 1}: "))
    complex_numbers.append(Complex(real, imag))

# Compute the sum of the complex numbers
result = add_complex_numbers(complex_numbers)

# Print the result
print(f"The sum of the complex numbers is: {result}")
```

In this code, the Complex class has two instance variables, real and imag, to represent the real and imaginary parts of the complex number. The __init__ method is used to initialize the instance variables when a Complex object is created. The __str__ method is used to generate a string representation of the complex number in the form real + imag i. The __add__ method

is used to implement the addition of two complex numbers. The `add_complex_numbers` function takes a list of `Complex` objects and returns their sum. The main part of the program reads the number of complex numbers from the console, reads each complex number, computes their sum using the `add_complex_numbers` function, and prints the result.

10. Develop a program that uses class Student which prompts the user to enter marks in three subjects and calculates total marks, percentage and displays the score card details. [Hint: Use list to store the marks in three subjects and total marks. Use `__init__()` method to initialize name, USN and the lists to store marks and total, Use `getMarks()` method to read marks into the list, and `display()` method to display the score card details.]

```
class Student:
    def __init__(self, name, usn):
        self.name = name
        self.usn = usn
        self.subjects = []
        self.total = 0
        self.percentage = 0

    def get_marks(self):
        for i in range(3):
            subject_mark = int(input(f"Enter the marks for subject {i + 1}: "))
            self.subjects.append(subject_mark)
            self.total += subject_mark

    def display(self):
        self.percentage = self.total / 3
        print("Name:", self.name)
        print("USN:", self.usn)
        print("Marks in subjects:", self.subjects)
        print("Total marks:", self.total)
        print("Percentage:", self.percentage)

# Read the student details
name = input("Enter the name of the student: ")
usn = input("Enter the USN of the student: ")

# Create a Student object
student = Student(name, usn)

# Get the marks
student.get_marks()

# Display the score card
student.display()
```

In this code, the Student class has five instance variables: name, usn, subjects, total, and percentage. The `__init__` method is used to initialize the instance variables when a Student object is created. The `get_marks` method prompts the user to enter the marks for three subjects and stores them in the subjects list and calculates the total marks. The display

method displays the student's name, USN, marks in each subject, total marks, and percentage. The main part of the program reads the student's name and USN from the console, creates a Student object, calls the `get_marks` method to get the marks, and then calls the `display` method to display the score card.